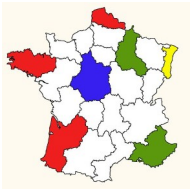
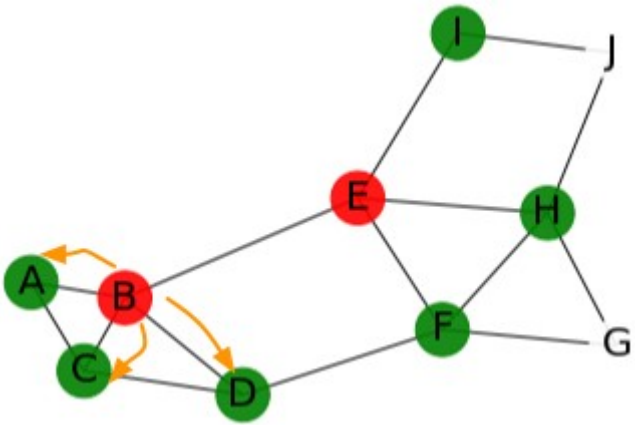
Nelson Mandela LYCÉE POLYVALENT • POITIERS	**T NSI - Numérique et Sciences de l'Informatique** **Structure de données : Les types abstraits : les graphes** **Séquence : Coloration d'un graphe**	

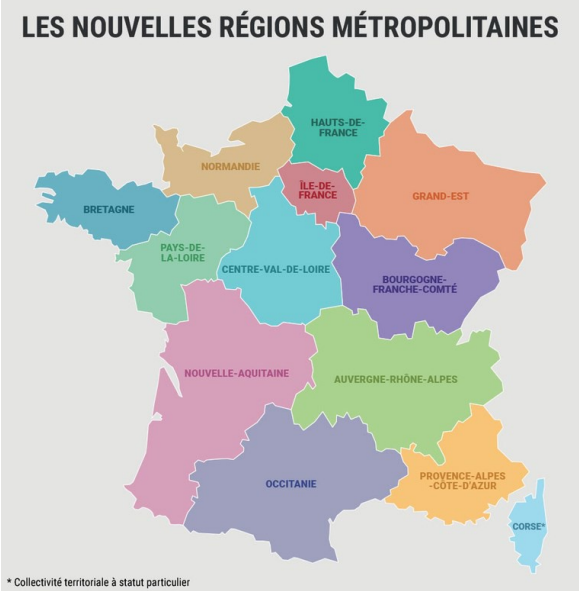
Objectifs :

- Découvrir les algorithmes de parcours de graphe
- Mettre en oeuvre un algorithme simple de coloration de graphe



Colorier un graphe, c'est associer une couleur à chaque sommet. Les problèmes de coloration consistent généralement à associer une couleur différente à chaque sommet adjacent, un peu comme la coloration d'une carte du monde où les pays voisins sont représentés avec une couleur différente pour les distinguer.

Quel est le nombre minimal de couleurs pour colorier un graphe ?



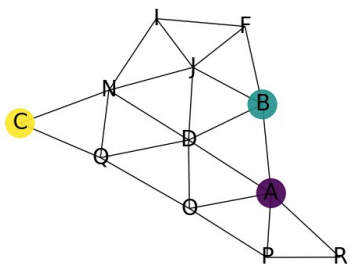
Ce problème n'est pas forcément simple et il a fallu utiliser l'informatique pour prouver le théorème des 4 couleurs : tout graphe peut se colorier avec un minimum de 4 couleurs. Nous allons ici voir un algorithme simple qui ne réalise pas forcément une coloration optimale du graphe,

Par la suite, on va travailler sur le graphe non-orienté des régions françaises. On donne ci-dessous le dictionnaire d'adjacence de ce graphe :

```
regions = {'Haut': {'Normandie', 'IdF','Est'},
'Normandie': {'Bretagne', 'Loire', 'Centre', 'IdF', 'Haut'},
'IdF': {'Normandie', 'Haut', 'Est', 'Bourgogne', 'Centre'},
'Est': {'Haut', 'IdF', 'Bourgogne'},
'Bretagne': {'Normandie', 'Loire'},
'Loire': {'Bretagne', 'Normandie', 'Centre', 'Aquitaine'},
'Centre': {'Loire', 'Normandie', 'IdF', 'Bourgogne', 'Auvergne', 'Aquitaine'},
'Bourgogne': {'Centre', 'IdF', 'Est', 'Auvergne'},
'Aquitaine': {'Loire', 'Centre', 'Auvergne', 'Occitanie'},
'Auvergne': {'Aquitaine', 'Centre', 'Bourgogne', 'Provence', 'Occitanie'},
'Occitanie': {'Aquitaine', 'Auvergne', 'Provence'},
'Provence': {'Occitanie', 'Auvergne'},
'Corse': {}}
```

Cela pourrait donner :

Mais l'idéal serait une carte animée



L'idée est de donner :

- Une animation des parcours en largeur et en longueur d'un graphe
- Une coloration par
  - un algo aléatoire
  - un algo glouton
  - un algo plus élaboré type Welsh-Powel ou DSATUR

voir l'animation sur https://hmalherbe.fr/thalesm/gestclasse/documents/Terminale_NSI/2021-2022/TP/Projets/Graphes/Coloration_graphes_cartes_geographiques.html

Exemple de l'algorithme de Welsh-Powel :

Les couleurs sont numérotées 1, 2, 3 ... (Les couleurs sont les naturels non nuls).

1) Dresser une liste des sommets (dans un ordre décroissant des degrés).

2) Tant que tous les sommets ne sont pas coloriés :

Choisir le premier sommet non encore colorié de la liste et lui affecter la plus petite couleur possible, non déjà affectée à un sommet adjacent).

- Remarque : En modifiant l'ordre des sommets de la liste, on peut parfois obtenir un nombre moindre de couleurs

Visualisation d'un graphe

Pour ce TP, nous allons utiliser la bibliothèque graphviz qui permet de créer, manipuler et visualiser simplement des graphes non-orienté (Graph) et orientés (Digraph).

```
from graphviz import Graph
import matplotlib.pyplot as plt
```

Les méthodes de base pour les graphes sont : Instruction	Effet
`g=Graph (engine=type)`	Crée une instance de graphe. `engine` est optionnel est peut valoir : circile, dot, fdp, neato, osage, patchwork, sfdp, twopi.
`g.node('sommet', 'label', fillcolor="#26c9b3")`	Créer un sommet. Le label est optionnel s'il n'est pas rempli le nom du sommet sera utilisé. `fillcolor` est optionnel et représente la couleur de remplissage (composantes RVB en hexadécimal). **Attention aux types de données, ce sont des types str**
`g.edge('sommet1', 'sommet2', 'poids')`	Créer une arête ou un arc entre deux sommets. Attention le poids est une chaine de caractère et est optionnel.
`print(g)`	Affiche le graphe en mode texte (type dictionnaire d'adjacence).
`g.view()`	Pour afficher le graphe en mode graphique. Dans ce cas il faut que l'exécutable de graphviz ait été installé. Celui-ci génère un fichier pdf qui est affiché avec le programme de visualisation par défaut du système.

Il y a aussi la bibliothèque **networkx**, que vous avez déjà utilisé. Voici un exemple d'utilisation :

```
import matplotlib.pyplot as plt
import networkx as nx
G = nx.Graph()
G.add_node("Auvergne-Rhône-Alpes",label="A",col=1)
G.add_node("Bourgogne-Franche-Comté",label="B",col=2)
G.add_node("Bretagne",label="C",col=3)
G.add_node("Centre-Val de Loire",label="D")
G.add_node("Corse",label="E")
G.add_node("Grand Est",label="F")
G.add_node("Guadeloupe",label="G")
```

Représenter le graphe à l'aide de la librairie networkx

```
# nodes
plt.clf()
nx.draw_networkx_nodes(G, pos, node_size=700,node_color=colorNodes,alpha=0.9)
# labels
nx.draw_networkx_labels(G, pos, labels=labels_nodes, \
                font_size=20, \
                font_color='black', \
                font_family='sans-serif')
# edges
nx.draw_networkx_edges(G, pos)
plt.axis('off')
plt.show()
```